

BÁO CÁO CẬP NHẬT TIẾN ĐỘ NGHIÊN CỨU

Lightweight Multi-task Model for Breast Ultrasound Images

Nhóm thực hiện

Phạm Nhật Duy - 25C15038
Lê Thanh Thái Quảng - 25C15058

Ngày 13 tháng 6 năm 2026

Tóm tắt cập nhật

So với proposal ban đầu, định hướng triển khai đã được điều chỉnh do hạn chế tài nguyên huấn luyện. Thay vì sử dụng các backbone tương đối nặng như ResNet-18 hoặc EfficientNet-B0 kết hợp nhiều thí nghiệm augmentation/policy, nhóm hiện chuyển sang xây dựng một mô hình lightweight cho ảnh siêu âm vú BUSI. Sự điều chỉnh này được định hướng thêm bởi paper [mednet.pdf](#), trong đó hướng tiếp cận MedNet cho thấy việc thiết kế mô hình phù hợp với miền ảnh y khoa là một cơ sở hợp lý hơn so với chỉ dùng trực tiếp các backbone tổng quát.

Mô hình hiện tại khai thác đồng thời hai nguồn tín hiệu của BUSI: nhãn phân loại ảnh ở cấp mẫu và mask tổn thương. Vì vậy, hướng triển khai được mở rộng từ classification đơn lẻ sang multi-task learning gồm classification và segmentation.

1. Thông tin thành viên

Bảng 1: Thông tin nhóm thực hiện

Thành viên	Mã số	Vai trò hiện tại
Phạm Nhật Duy	25C15038	Xây dựng pipeline xử lý dữ liệu, huấn luyện mô hình và phân tích kết quả thực nghiệm.
Lê Thanh Thái Quảng	25C15058	Khảo sát tài liệu, thiết kế hướng tiếp cận, kiểm tra kết quả và chuẩn bị báo cáo.

2. Bài toán nghiên cứu

Proposal ban đầu tập trung vào câu hỏi: các chiến lược data augmentation nào giúp cải thiện hiệu năng phân loại ảnh siêu âm vú, và augmentation nào có thể gây hại do làm mất hoặc làm sai lệch vùng tổn thương.

Vai trò của paper MedNet trong điều chỉnh hướng nghiên cứu

Paper `mednet.pdf` là cơ sở chính cho sự chuyển hướng từ proposal ban đầu sang bản triển khai hiện tại. Nếu proposal ban đầu đặt trọng tâm vào ResNet-18/EfficientNet-B0 và so sánh nhiều augmentation policy, thì paper MedNet gợi ý một hướng phù hợp hơn với bối cảnh hiện tại: xây dựng mô hình dành cho ảnh y khoa, ưu tiên khả năng học đặc trưng trong miền medical imaging và giảm phụ thuộc vào các backbone tổng quát vốn có thể đòi hỏi nhiều tài nguyên huấn luyện.

Từ cơ sở đó, nhóm không xem việc chuyển hướng là thay đổi đề tài độc lập, mà là điều chỉnh có căn cứ: vẫn giữ bài toán ảnh siêu âm vú BUSI, nhưng chuyển trọng tâm từ “so sánh augmentation trên backbone nặng” sang “xây dựng lightweight medical model có thể học đồng thời classification và segmentation”. Phần augmentation từ proposal vẫn được giữ lại như một hướng cải tiến phụ sau khi baseline MedNet/multi-task ổn định.

Điều chỉnh bài toán hiện tại

Sau khi triển khai thử và đối chiếu với tài nguyên huấn luyện thực tế, nhóm điều chỉnh trọng tâm sang bài toán xây dựng mô hình nhẹ cho BUSI. Mục tiêu hiện tại là:

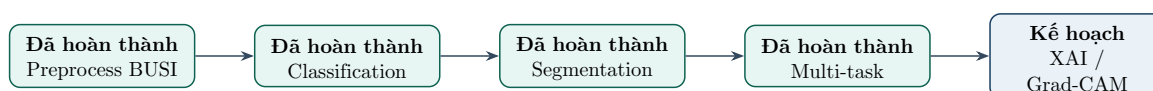
- phân loại ảnh siêu âm vú thành ba lớp *benign*, *malignant*, *normal*;
- phân đoạn vùng tổn thương dựa trên mask đi kèm;
- huấn luyện multi-task model dùng chung backbone để tận dụng đồng thời thông tin classification và segmentation.

Việc thay đổi này vẫn giữ tinh thần chính của đề tài là xử lý ảnh siêu âm y khoa một cách có kiểm soát, nhưng chuyển trọng tâm từ đánh giá nhiều augmentation sang tối ưu kiến trúc và pipeline phù hợp với tài nguyên hiện có.

Do đó, câu hỏi nghiên cứu ở giai đoạn hiện tại được xác định lại theo hướng: *một mô hình lightweight multi-task có thể khai thác đồng thời nhãn phân loại và mask tổn thương của BUSI đến mức nào trong điều kiện tài nguyên huấn luyện hạn chế?* Phần augmentation không bị loại bỏ hoàn toàn, nhưng được chuyển thành hướng kiểm tra phụ sau khi baseline multi-task đủ ổn định.

3. Tiến độ thực hiện và kế hoạch triển khai

3.1 Đối chiếu với kế hoạch tổng thể



Hình 1: Tiến độ triển khai hiện tại sau khi điều chỉnh phạm vi.

Bảng 2: Trạng thái các hạng mục triển khai

Hạng mục	Trạng thái	Ghi chú
Chuẩn bị dữ liệu BUSI	Hoàn thành	Đã có cấu trúc <code>train/val/test</code> theo lớp, gồm <code>images/</code> và <code>masks/</code> .
Classification baseline	Hoàn thành	Đã train mô hình <code>MedNet</code> cho ba lớp BUSI.
Segmentation model	Hoàn thành bước đầu	Đã train <code>MedNetSegmentation</code> ; kết quả test còn cần cải thiện.
Multi-task model	Hoàn thành bước đầu	Đã train <code>MedNetMultiTask</code> với shared backbone và hai output.
Test trực quan	Hoàn thành bước đầu	Đã xuất ảnh dự đoán cho classification, segmentation và multi-task.
XAI bằng Grad-CAM	Chưa triển khai	Sẽ bổ sung để giải thích vùng ảnh ảnh hưởng đến quyết định classification.

4. Chi tiết phần hiện tại

4.1 Dữ liệu và tiền xử lý

Pipeline hiện tại sử dụng dataset BUSI đã được chuẩn hóa thành thư mục ảnh. Mỗi split gồm ba lớp *benign*, *malignant*, *normal*. Với mỗi lớp, dữ liệu được tách thành ảnh gốc và mask:

- `data/busi/train/<class>/images/`
- `data/busi/train/<class>/masks/`
- cấu trúc tương tự cho `val/` và `test/`.

Bảng 3: Phân bố dữ liệu BUSI sau khi preprocess

Split	Benign	Malignant	Normal	Tổng
Train	305	147	93	545
Validation	65	31	19	115
Test	67	32	21	120

Phân bố trên cho thấy dữ liệu không cân bằng hoàn toàn, trong đó lớp *benign* chiếm tỷ lệ lớn hơn *malignant* và *normal*. Đây là lý do nhóm dùng Focal Loss cho nhánh classification. Tuy nhiên, ở giai đoạn hiện tại nhóm chưa có thí nghiệm đối chứng giữa Focal Loss và Cross Entropy, nên lựa chọn này được xem là một giả thuyết kỹ thuật hợp lý cần được kiểm chứng thêm trong báo cáo cuối.

4.2 Model construction

Mô hình hiện tại không dùng ResNet/EfficientNet như proposal ban đầu. Thay vào đó, nhóm cài đặt backbone nhẹ trong `model.py`:

- **Depthwise Separable Convolution:** giảm số tham số và chi phí tính toán so với convolution chuẩn.
- **Residual connection:** giữ khả năng học đặc trưng ổn định khi tăng số tầng.
- **CBAM attention:** bổ sung channel attention và spatial attention để tập trung vào đặc trưng quan trọng.
- **Shared backbone:** dùng chung encoder cho classification và segmentation trong multi-task model.

Bước 1: Xây dựng loss cho classification

Classification branch sử dụng Focal Loss thay vì Cross Entropy thông thường. Lý do là BUSI có thể bị mất cân bằng giữa các lớp, trong đó số lượng ảnh *normal*, *benign* và *malignant* không hoàn toàn đồng đều. Focal Loss giúp mô hình tập trung hơn vào các mẫu khó bằng cách giảm ảnh hưởng của những mẫu đã được phân loại dễ.

$$\mathcal{L}_{focal} = (1 - p_t)^\gamma \mathcal{L}_{CE}$$

Trong code, thành phần này được cài đặt trong lớp `FocalLoss`. Tham số $\gamma = 2$ được dùng làm mặc định.

Bước 2: Xây dựng attention module

Backbone hiện tại có thể bật hoặc tắt CBAM. CBAM gồm hai phần:

- **Channel Attention:** học trọng số theo kênh đặc trưng, dùng cả average pooling và max pooling trên không gian ảnh.
- **Spatial Attention:** học vùng không gian quan trọng trong feature map, dùng average/max theo chiều kênh rồi đưa qua convolution 7×7 .

Mục tiêu của attention là giúp mô hình tập trung vào vùng ảnh có thông tin y khoa quan trọng hơn, thay vì học phân bố đặc trưng một cách hoàn toàn đồng đều trên toàn ảnh.

Bước 3: Xây dựng convolution block nhẹ

Mỗi block chính dùng *depthwise separable convolution*. Thay vì dùng một convolution chuẩn để học đồng thời không gian và kênh, block này tách thành:

1. **Depthwise convolution:** convolution riêng trên từng kênh đầu vào.
2. **Pointwise convolution:** convolution 1×1 để trộn thông tin giữa các kênh.
3. **Batch normalization và ReLU:** ổn định huấn luyện và thêm phi tuyến.

Cách thiết kế này giúp giảm chi phí tính toán so với ResNet-style convolution chuẩn, phù hợp hơn với giới hạn tài nguyên huấn luyện hiện tại.

Bảng 4: Kích thước mô hình hiện tại

Model	Số tham số	Kích thước checkpoint	Ghi chú
MedNet classification	3.27M	13.18 MB	Backbone nhẹ + classification head.
MedNetSegmentation	15.54M	62.31 MB	Backbone nhẹ + decoder segmentation.
MedNetMultiTask	15.81M	63.37 MB	Shared backbone + hai nhánh output.

Các số liệu trên giúp cụ thể hóa nhận định “lightweight” ở mức số tham số và kích thước checkpoint. Dù vậy, nhóm vẫn cần bổ sung FLOPs/MACs và thời gian inference để so sánh chặt hơn với ResNet-18, EfficientNet-B0 hoặc các baseline nhẹ như MobileNetV2.

Bước 4: Xây dựng residual backbone

Backbone ResidualCBAMBackbone gồm 5 stage:

Bảng 5: Các stage trong lightweight backbone

Stage	Input channel	Output channel	Vai trò
Stage 1	3	64	Trích đặc trưng mức thấp từ ảnh RGB.
Stage 2	64	128	Giảm kích thước không gian, tăng số kênh đặc trưng.
Stage 3	128	256	Học đặc trưng trung gian.
Stage 4	256	512	Học đặc trưng mức cao, giữ lại skip feature cho segmentation.
Stage 5	512	1024	Tạo encoded feature dùng cho classification và decoder.

Mỗi stage trả về activation trung gian. Các activation này được dùng làm skip connection cho segmentation decoder, tương tự tinh thần encoder-decoder nhưng nhẹ hơn so với U-Net/ResNet backbone đầy đủ.

Bước 5: Xây dựng classification head

Classification head được dùng trong MedNet và MedNetMultiTask. Sau khi backbone tạo encoded feature có 1024 kênh, model thực hiện:

- 1. Adaptive average pooling để đưa feature map về vector đặc trưng toàn cục.

2. Flatten vector.
3. Fully connected layer từ 1024 xuống 256 chiều.
4. Dropout 0.4 để giảm overfitting.
5. Fully connected layer cuối để xuất logits cho 3 lớp BUSI.

Output của nhánh này là $\mathbf{z}_{cls} \in \mathbb{R}^3$, tương ứng với ba lớp *benign*, *malignant*, *normal*.

Bước 6: Xây dựng segmentation decoder

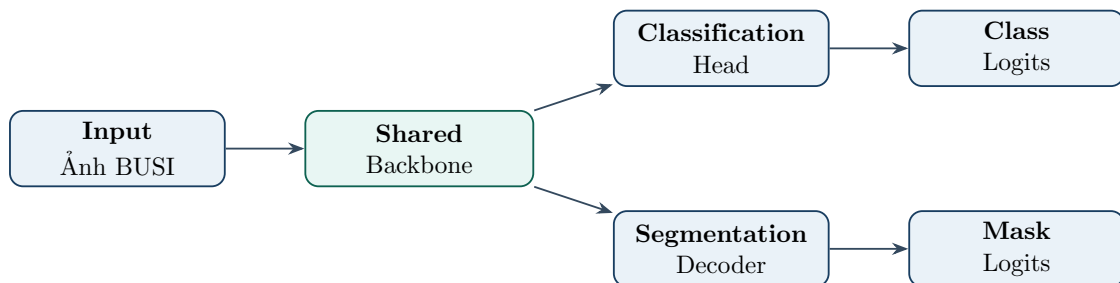
Segmentation branch được dùng trong `MedNetSegmentation` và `MedNetMultiTask`. Decoder gồm 4 block:

- Decoder 4: kết hợp feature stage 5 với skip feature stage 4.
- Decoder 3: kết hợp output trước đó với skip feature stage 3.
- Decoder 2: kết hợp output trước đó với skip feature stage 2.
- Decoder 1: kết hợp output trước đó với skip feature stage 1.

Mỗi decoder block thực hiện upsample bằng bilinear interpolation, concatenate với skip feature, sau đó dùng hai convolution 3×3 . Cuối cùng, segmentation head dùng convolution 1×1 để tạo mask logit một kênh. Logit được resize lại về kích thước ảnh đầu vào.

Bước 7: Xây dựng multi-task model

Model `MedNetMultiTask` dùng một shared backbone và hai output song song:



Hình 2: Kiến trúc multi-task model hiện tại.

Thiết kế này giúp classification và segmentation cùng cập nhật backbone. Classification học đặc trưng toàn cục để phân biệt loại ảnh, còn segmentation buộc backbone giữ lại thông tin không gian về vùng tổn thương.

4.3 Các task đã triển khai

Phần triển khai hiện tại không chỉ dừng ở việc chạy từng script độc lập, mà đã hình thành một pipeline thử nghiệm có ba tầng. Tầng thứ nhất là classification để giữ lại mục tiêu ban đầu của proposal: phân loại ảnh siêu âm vú. Tầng thứ hai là segmentation để tận dụng mask có sẵn trong BUSI và kiểm tra khả năng định vị vùng tổn thương. Tầng thứ ba là multi-task learning, tức là đưa hai bài toán này vào cùng một mô hình nhẹ dùng chung backbone.

Bảng 6: Ý tưởng và mức độ hoàn thành theo từng task

Task	Ý tưởng đã thực hiện	Kết quả hiện tại
Classification	Xây dựng baseline nhẹ để phân loại ảnh BUSI thành <i>benign</i> , <i>malignant</i> , <i>normal</i> . Đây là phần bám sát nhất với proposal ban đầu.	Đã train được MedNet ; có checkpoint, epoch log, metric test và ảnh dự đoán trực quan.
Segmentation	Tận dụng mask đi kèm trong BUSI để học vùng tổn thương. Mục tiêu là bổ sung khả năng định vị, không chỉ dự đoán nhãn ảnh.	Đã train được MedNetSegmentation ; pipeline chạy end-to-end nhưng mask dự đoán còn chưa ổn định.
Multi-task	Kết hợp classification và segmentation trong một mô hình shared-backbone. Classification học ngữ nghĩa toàn cục, segmentation giữ thông tin không gian vùng bệnh.	Đã train được MedNetMultiTask ; classification cao hơn baseline trong lần chạy hiện tại, segmentation ở mức bước đầu.
Visual testing	Sinh ảnh test để kiểm tra mô hình bằng mắt, gồm nhãn thật, nhãn dự đoán, confidence, mask thật và mask dự đoán.	Đã xuất ảnh PNG cho cả ba task trong thư mục outputs/busi .

Ý nghĩa của phần đã triển khai

Điểm quan trọng của giai đoạn hiện tại là nhóm đã chuyển được ý tưởng từ một pipeline classification đơn lẻ sang một pipeline học đa nhiệm có thể kiểm tra cả hai khía cạnh: mô hình dự đoán bệnh gì và mô hình có học được vùng tổn thương hay không. Đây là nền tảng cần thiết trước khi bổ sung Grad-CAM/XAI, vì XAI sẽ giúp trả lời tiếp câu hỏi mô hình classification đang nhìn vào vùng nào khi đưa ra quyết định.

Các file chính tương ứng với ba task:

- `train_busi_classification.py` và `test_busi_classification.py`: train/test classification.
- `train_busi_segmentation.py` và `test_busi_segmentation.py`: train/test segmentation.
- `train_busi_multi.py` và `test_busi_multi.py`: train/test multi-task.

4.4 Quy trình chạy end-to-end

Toàn bộ phần hiện tại có thể được hiểu như một pipeline gồm các bước nối tiếp nhau từ dữ liệu thô/NPZ đến checkpoint và ảnh test trực quan:

Bảng 7: Pipeline chạy model hiện tại

Bước	Lệnh / module	Kết quả
1	<code>uv run preprocess.py -dataset busi -overwrite</code>	Tạo cấu trúc <code>data/busi/train</code> , <code>data/busi/val</code> , <code>data/busi/test</code> .
2	<code>uv run train_busi_classification.py</code>	Train classifier và lưu checkpoint classification.
3	<code>uv run train_busi_segmentation.py</code>	Train segmentation model và lưu checkpoint segmentation.
4	<code>uv run train_busi_multi.py</code>	Train shared-backbone multi-task model.
5	<code>uv run test_busi_classification.py</code>	Xuất ảnh test có nhãn thật, nhãn dự đoán và confidence.
6	<code>uv run test_busi_segmentation.py</code>	Xuất panel so sánh ảnh gốc, mask thật và mask dự đoán.
7	<code>uv run test_busi_multi.py</code>	Xuất panel kết hợp classification và segmentation.

Với multi-task learning, loss hiện tại được tính theo công thức:

$$\mathcal{L}_{total} = \mathcal{L}_{classification} + \lambda \mathcal{L}_{segmentation}$$

Trong đó $\mathcal{L}_{classification}$ là Focal Loss, $\mathcal{L}_{segmentation}$ là BCEWithLogitsLoss và λ là trọng số segmentation loss.

4.5 Training process

Bước 1: Chuẩn bị dữ liệu train/val/test

Sau khi preprocess, các script train đọc dữ liệu từ `data/busi`. Tùy task, dataset class sẽ trả về:

- Classification: $(image, label)$.
- Segmentation: $(image, mask)$.
- Multi-task: $(image, label, mask)$.

Ảnh được chuyển sang RGB, mask được chuyển sang grayscale. Với segmentation và multi-task, mask được chuẩn hóa về $[0, 1]$, thêm chiều channel và threshold thành binary mask.

Bước 2: Augmentation và normalization

Train split dùng augmentation nhẹ:

- resize về 224×224 ;
- horizontal flip và vertical flip;
- random brightness/contrast;

- shift, scale và rotate với mức vừa phải;
- normalize theo mean/std ImageNet;
- chuyển sang tensor bằng `ToTensorV2`.

Validation và test split chỉ dùng resize, normalize và tensor conversion. Với segmentation/multi-task, Albumentations được dùng để áp dụng cùng phép biến đổi cho ảnh và mask.

Bước 3: Khởi tạo mô hình và cấu hình train

Các cấu hình chính hiện tại:

Bảng 8: Cấu hình huấn luyện mặc định

Task	Batch size	Loss / metric chính
Classification	32	Focal Loss; Accuracy, AUC.
Segmentation	16	BCEWithLogitsLoss; Dice, IoU.
Multi-task	8	Focal Loss + λ BCEWithLogitsLoss; Accuracy, AUC, Dice, IoU.

Optimizer đang dùng là AdamW với learning rate 3×10^{-4} và weight decay 10^{-4} . Scheduler là CosineAnnealingLR với $\eta_{min} = 10^{-6}$. Early stopping được dùng để dừng khi validation metric không cải thiện đủ sau một số epoch.

Bước 4: Training loop

Quy trình train mỗi epoch gồm:

1. Đưa model sang train mode.
2. Duyệt từng batch trong train loader.
3. Đưa image, label và/hoặc mask lên device.
4. Forward qua model để lấy logits.
5. Tính loss theo task.
6. Backpropagation và optimizer step.
7. Tính metric train.
8. Chạy validation không cập nhật gradient.
9. Cập nhật scheduler.
10. Lưu checkpoint nếu validation metric tốt hơn trước đó.
11. Ghi lại epoch log vào `epoch_log.csv`.

Với multi-task, một batch đi qua model sẽ trả về hai output:

$$(\mathbf{z}_{cls}, \mathbf{z}_{seg}) = f_{\theta}(x)$$

Trong đó $\mathbf{z}_{cls}$ là classification logits và $\mathbf{z}_{seg}$ là segmentation logits. Tổng loss hiện dùng:

$$\mathcal{L}_{total} = \mathcal{L}_{focal}(\mathbf{z}_{cls}, y) + \lambda \mathcal{L}_{BCE}(\mathbf{z}_{seg}, m)$$

Hiện tại $\lambda = 1.0$. Checkpoint multi-task đang được chọn theo validation accuracy, trong khi Dice/IoU vẫn được ghi lại để đánh giá segmentation.

Bước 5: Lưu kết quả train

Sau mỗi lần train, các artifact chính được lưu gồm:

- `best_model.pt`: checkpoint tốt nhất theo validation metric.
- `epoch_log.csv`: log theo từng epoch.
- `result.csv`: kết quả tổng hợp trên test set.

4.6 Testing and inference process

Bước 1: Load checkpoint

Các script `test_busi_*.py` khởi tạo đúng kiến trúc model tương ứng, sau đó load checkpoint `best_model.pt`. Model được chuyển sang eval mode để chạy inference.

Bước 2: Chọn mẫu test

Hàm tiện ích trong `test_busi_utils.py` lấy mẫu từ test split theo cách cân bằng giữa ba lớp. Cách này giúp ảnh trực quan không bị lệch hoàn toàn về một lớp.

Bước 3: Chuẩn bị input

Ảnh test được resize về 224×224 , scale về $[0, 1]$, normalize theo ImageNet mean/std rồi chuyển thành tensor có shape $(1, 3, 224, 224)$. Với segmentation, mask thật được resize bằng nearest neighbor để giữ nhãn nhị phân.

Bước 4: Inference theo từng task

Bảng 9: Quy trình inference theo task

Task	Output model	Xử lý sau inference
Classification	Class logits	Softmax, lấy predicted class và confidence.
Segmentation	Mask logits	Sigmoid, threshold 0.5 để tạo binary mask, tính Dice với mask thật.
Multi-task	Class logits và mask logits	Kết hợp hai bước trên: dự đoán lớp, confidence, mask và Dice.

Với ảnh *normal*, mask thật có thể rỗng hoàn toàn. Cách tính Dice/IoU hiện tại dùng hệ số làm tròn 10^{-6} : nếu ground-truth rỗng và prediction cũng rỗng thì Dice gần 1; nếu ground-truth rỗng nhưng prediction có vùng dương tính giả thì Dice gần 0. Điều này giúp phản ánh lỗi false positive trên ảnh *normal*, nhưng nhóm vẫn cần tách riêng thống kê lỗi này trong phân tích cuối kỳ.

Bước 5: Xuất ảnh trực quan

Kết quả test được lưu thành PNG:

- Classification: một panel gồm ảnh gốc, nhãn thật, nhãn dự đoán và confidence.
- Segmentation: ba panel gồm ảnh gốc, mask thật và mask dự đoán kèm Dice.
- Multi-task: ba panel gồm ảnh gốc kèm classification, mask thật và mask dự đoán kèm Dice.

Đây là phần quan trọng để kiểm tra trực quan xem model có dự đoán hợp lý hay không, đặc biệt với segmentation vì metric tổng quát chưa phản ánh hết lỗi về hình dạng và vị trí vùng tổn thương.

5. Kết quả output hiện tại

5.1 Kết quả định lượng

Bảng 10: Kết quả test hiện tại trên BUSI

Task	Val tốt nhất	Test Acc	Test AUC	Test Dice	Test IoU
Classification	70.43% Acc	72.50%	0.8678	–	–
Segmentation	0.6000 Dice	–	–	0.6422	0.5425
Multi-task	74.78% Acc	81.67%	0.9129	0.6339	0.5494

AUC trong bảng được tính theo chiến lược multi-class one-vs-rest như trong code hiện tại. Với ba lớp *benign*, *malignant*, *normal*, giá trị AUC phản ánh mức phân biệt tổng quát giữa từng lớp và phần còn lại, nhưng chưa thay thế được per-class recall/F1, đặc biệt đối với lớp *malignant*.

Đọc kết quả một cách thận trọng

Trong lần chạy hiện tại, multi-task đạt Test Acc và Test AUC cao hơn classification-only. Tuy nhiên, nhóm chưa xem đây là kết luận chắc chắn rằng multi-task luôn tốt hơn baseline, vì hiện mới có một seed và chưa có kiểm định độ ổn định. Để kết luận mạnh hơn, cần chạy lại cùng split với nhiều seed, cùng cấu hình preprocessing/early stopping và bổ sung confusion matrix, Macro-F1, sensitivity/specificity theo từng lớp.

Nhận xét về segmentation

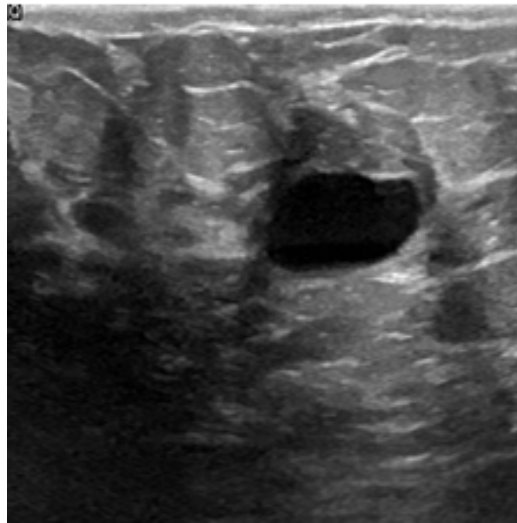
Kết quả segmentation hiện tại đã chạy được end-to-end và có Dice/IoU ở mức bước đầu. Tuy nhiên, khi kiểm tra trực quan, mask dự đoán chưa thực sự ổn định và vẫn cần cải thiện thêm, đặc biệt ở các trường hợp vùng tổn thương nhỏ, biên không rõ hoặc ảnh có nhiễu siêu âm mạnh.

Bảng 11: Các bằng chứng thực nghiệm còn thiếu để củng cố kết luận			
Nội dung cần bổ sung	Lý do cần thiết	Trạng thái hiện tại	
Confusion matrix	Kiểm tra mô hình nhầm lớp nào, đặc biệt có bỏ sót <i>ma-lignant</i> hay không.	Chưa đưa vào report hiện tại.	
Per-class precision/recall/F1	Accuracy tổng quát có thể che khuất lỗi ở lớp ít mẫu.	Đưa vào kế hoạch cải tiến gần nhất.	
Sensitivity/specificity	Phù hợp hơn với yêu cầu đánh giá trong bài toán y khoa.	Chưa triển khai trong metric output.	
Nhiều seed	Kiểm tra độ ổn định trên dataset nhỏ như BUSI.	Hiện mới báo cáo một lần chạy.	
Ablation loss/baseline	Kiểm chứng Focal Loss, CBAM và multi-task có đóng góp thật sự hay không.	Chưa hoàn thành.	

5.2 Kết quả trực quan

Các ảnh test đã được xuất ra trong thư mục **outputs**. Trong report này, nhóm chọn các ví dụ đại diện để minh họa kết quả hiện tại. Các hình không chỉ dùng để trình bày output, mà còn giúp đánh giá nhanh tính hợp lý của mô hình: classification có dự đoán đúng nhãn hay không, segmentation có bám vùng tổn thương hay không, và multi-task có giữ được cả hai loại thông tin hay không.

true: benign
predict benign
confidence: 0.4554



Hình 3: Ví dụ output classification. Mô hình dự đoán đúng lớp *benign*; confidence còn tương đối thấp, cho thấy mô hình vẫn chưa thật sự chắc chắn ở một số mẫu.

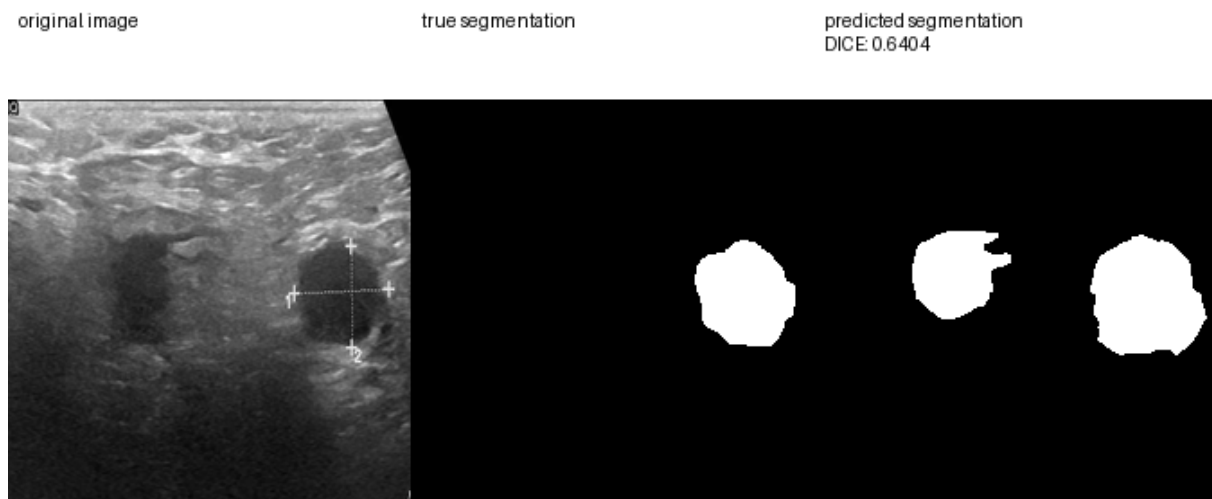
original image

true segmentation

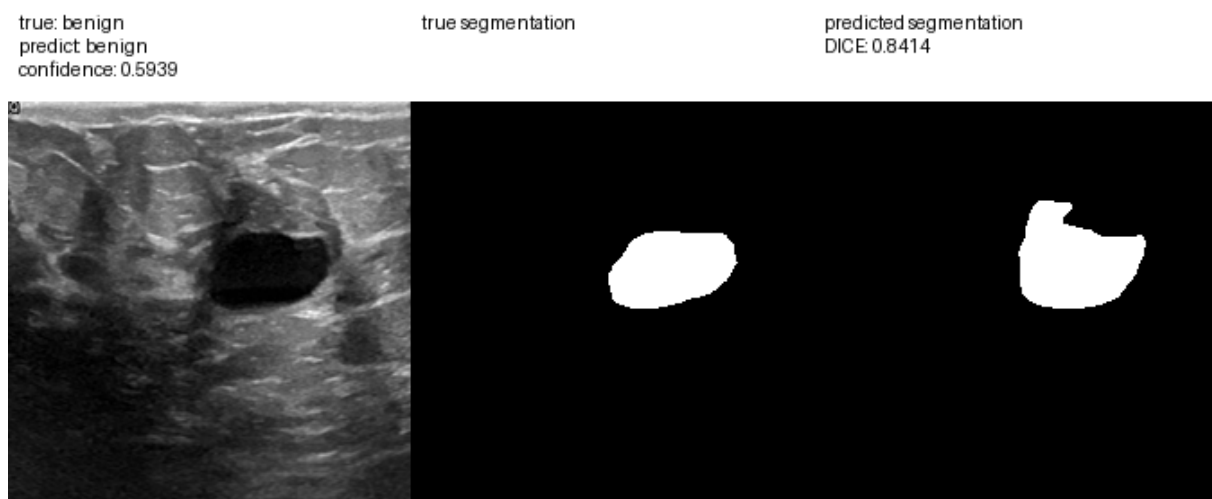
predicted segmentation
DICE 0.8113



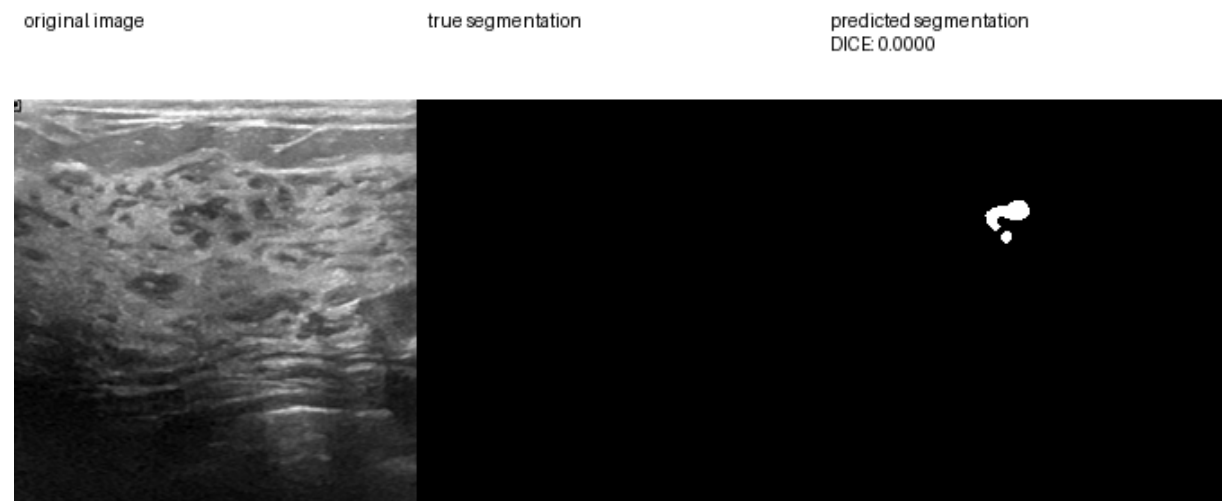
Hình 4: Ví dụ segmentation tương đối tốt trên mẫu benign. Mask dự đoán gần với mask thật và đạt Dice 0.8113.



Hình 5: Ví dụ segmentation trên mẫu malignant. Mô hình xác định được vùng tổn thương chính nhưng dự đoán bị tách thành nhiều vùng, Dice 0.6404. Đây là một minh chứng cho việc segmentation đã học được tín hiệu bước đầu nhưng còn cần cải thiện về hình dạng và tính liên mạch của mask.



Hình 6: Ví dụ output multi-task. Cùng một ảnh đầu vào, mô hình vừa dự đoán đúng lớp *benign*, vừa tạo mask segmentation với Dice 0.8414.



Hình 7: Ví dụ hạn chế hiện tại trên mẫu normal. Mask thật không có vùng tổn thương nhưng mô hình vẫn sinh một vùng dự đoán nhỏ, Dice 0.0000. Trường hợp này cho thấy cần cải thiện khả năng xử lý ảnh normal và giảm false positive.

Nhìn chung, các hình trên cho thấy pipeline đã tạo được output trực quan đầy đủ. Tuy nhiên, kết quả segmentation chưa đồng đều giữa các mẫu: có trường hợp Dice cao, có trường hợp mask bị tách vùng hoặc sinh false positive. Vì vậy, ở giai đoạn tiếp theo nhóm cần tập trung vào loss segmentation, threshold, cân bằng dữ liệu/mask và phân tích trực quan có hệ thống hơn.

Các hình được sử dụng trực tiếp trong report đã được đóng gói riêng tại `report_images/`. Các thư mục output đầy đủ vẫn được giữ lại để kiểm tra thêm:

- `report_images/`
- `lightweight-medical-model/outputs/busi/classification/img_224/test_images/`
- `lightweight-medical-model/outputs/busi/segmentation/img_224/test_images/`
- `lightweight-medical-model/outputs/busi/multi/img_224/test_images/`

5.3 Artifact đã tạo

Bảng 12: Artifact chính đã có

Task	Artifact
Classification	<code>lightweight-medical-model/outputs/busi/classification/img_224/cbam/run_1/best_model.pt</code>
Segmentation	<code>lightweight-medical-model/outputs/busi/segmentation/img_224/cbam/best_model.pt</code>
Multi-task	<code>lightweight-medical-model/outputs/busi/multi/img_224/cbam/seg_weight_1/best_model.pt</code>
Logs và metrics	<code>lightweight-medical-model/outputs/busi/*/result.csv</code> và <code>epoch_log.csv</code> tương ứng.

6. Cải tiến dự kiến

Hướng cải tiến tiếp theo

Các bước tiếp theo tập trung vào nâng chất lượng segmentation, tăng độ tin cậy của classification và bổ sung khả năng giải thích mô hình.

- G1. Cải thiện mô hình và quá trình huấn luyện.** Nhóm sẽ ưu tiên nâng chất lượng segmentation và tối ưu multi-task model. Cụ thể, phần segmentation sẽ được thử thêm Dice Loss hoặc Dice + BCE Loss, tinh chỉnh threshold, kiểm tra class/mask imbalance và xử lý tốt hơn các ảnh *normal* có mask rỗng. Với multi-task learning, nhóm sẽ thử các giá trị λ khác nhau cho segmentation loss và xem xét checkpoint theo cả validation accuracy lẫn validation Dice, thay vì chỉ ưu tiên classification.
- G2. Củng cố đánh giá thực nghiệm.** Để kết quả có sức thuyết phục hơn, nhóm sẽ bổ sung confusion matrix, per-class precision/recall/F1, Macro-F1, sensitivity và specificity, trong đó ưu tiên kiểm tra riêng lớp *malignant*. Ngoài ra, classification-only và multi-task sẽ được chạy lại với nhiều seed trên cùng split để báo cáo mean $\pm$ std, tránh kết luận dựa trên một lần chạy. Nhóm cũng sẽ bổ sung baseline/ablation như Cross Entropy so với Focal Loss, tắt/bật CBAM, thay đổi λ , và nếu đủ tài nguyên thì thêm ResNet-18, MobileNetV2 hoặc U-Net nhỏ.
- G3. Làm rõ tính lightweight của mô hình.** Bên cạnh số tham số và kích thước checkpoint đã báo cáo, nhóm sẽ bổ sung FLOPs/MACs, thời gian inference và so sánh với một số baseline chuẩn. Mục tiêu là biến nhận định “lightweight” từ mô tả định tính thành bằng chứng định lượng có thể đối chiếu.
- G4. Bổ sung XAI và phân tích trực quan.** Nhóm sẽ triển khai Grad-CAM cho nhánh classification để kiểm tra mô hình có tập trung vào vùng tổn thương hay không. Các kết quả trực quan sẽ được tổ chức theo nhóm mẫu đúng/sai, từng lớp bệnh, ảnh gốc, mask thật, mask dự đoán và Grad-CAM. Nếu Grad-CAM lệch khỏi mask tổn thương, nhóm sẽ phân tích khả năng mô hình đang học shortcut từ texture, nhiễu hoặc artifact trong ảnh siêu âm.
- G5. Áp dụng thêm data augmentation theo hướng proposal.** Sau khi baseline multi-task ổn định, nhóm sẽ quay lại khai thác ý tưởng augmentation từ proposal/paper ban đầu, nhưng chỉ chọn một số phương pháp phù hợp với tài nguyên hiện có thay vì so sánh toàn bộ augmentation policy. Các phép ưu tiên gồm rotation nhỏ, brightness/contrast nhẹ, shift/scale vừa phải và speckle noise. Với segmentation/multi-task, augmentation phải được áp dụng đồng bộ cho ảnh và mask để tránh làm lệch vùng tổn thương.

7. Kết luận cập nhật

Tính đến thời điểm hiện tại, nhóm đang ở giai đoạn **hoàn thành prototype end-to-end** của hướng triển khai mới. Nói cách khác, phần cốt lõi để chứng minh pipeline chạy được đã hoàn thành: dữ liệu BUSI đã được preprocess, mô hình lightweight đã được xây dựng, ba task classification/segmentation/multi-task đã được huấn luyện và các script test đã xuất được ảnh trực quan.

Bảng 13: Vị trí hiện tại trong plan tổng thể

Mốc	Mục tiêu	Trạng thái hiện tại
Mốc 1	Chuẩn bị dữ liệu và baseline chạy được.	Đã hoàn thành: dữ liệu BUSI có đủ train/val/test, classification baseline đã có kết quả.
Mốc 2	Mở rộng sang segmentation để tận dụng mask.	Đã hoàn thành bước đầu: segmentation chạy được nhưng chất lượng mask chưa ổn định.
Mốc 3	Kết hợp classification và segmentation bằng multi-task learning.	Đã hoàn thành prototype: multi-task có kết quả classification cao hơn baseline trong lần chạy hiện tại, segmentation ở mức cần cải thiện.
Mốc 4	Phân tích sâu, cải thiện model và bổ sung XAI.	Đang chuẩn bị triển khai: Grad-CAM, cải thiện segmentation loss, thêm metric y khoa.
Mốc 5	Hoàn thiện báo cáo cuối và đánh giá hệ thống.	Chưa hoàn thành: cần thêm thí nghiệm, phân tích lỗi và trực quan hóa giải thích.

So với proposal ban đầu, phạm vi đã được điều chỉnh do giới hạn tài nguyên huấn luyện: thay vì tập trung vào ResNet/EfficientNet và nhiều augmentation policy, nhóm chuyển sang lightweight multi-task model để tận dụng cả nhãn classification và mask segmentation. Paper `mednet.pdf` đóng vai trò cầu nối cho thay đổi này, vì cung cấp cơ sở để ưu tiên một mô hình định hướng cho ảnh y khoa thay vì chỉ dựa trên backbone tổng quát. Đây không phải là thay đổi rời khỏi bài toán ảnh siêu âm y khoa, mà là điều chỉnh để phù hợp hơn với tài nguyên thực tế và khai thác đầy đủ hơn dữ liệu BUSI.

Kết quả multi-task hiện tại là tín hiệu ban đầu đáng chú ý khi đạt Test Acc 81.67% và Test AUC 0.9129 trong một lần chạy. Tuy nhiên, nhóm chưa xem đây là bằng chứng cuối cùng rằng multi-task vượt trội hơn classification-only, vì chưa có nhiều seed, confusion matrix, per-class metric và kiểm định tính ổn định. Phần segmentation vẫn là điểm yếu chính: một số mẫu cho Dice tốt, nhưng một số mẫu normal hoặc biên tổn thương không rõ vẫn tạo false positive hoặc mask chưa đúng hình dạng. Vì vậy, giai đoạn tiếp theo sẽ không chỉ là chạy thêm model, mà là **phân tích lỗi có hệ thống**: xem mô hình sai ở nhóm ảnh nào, mask sai theo kiểu nào, lớp *malignant* có bị bỏ sót hay không, và Grad-CAM có tập trung vào vùng tổn thương hay không.

Tóm lại, nhóm hiện đã đi qua giai đoạn “xây pipeline và chứng minh chạy được”, đang bước vào giai đoạn “củng cố bằng chứng thực nghiệm”. Trọng tâm gần nhất là biến kết quả prototype thành kết quả nghiên cứu có thể bảo vệ trước phản biện: so sánh công bằng hơn, thêm per-class metric, đánh giá nhiều seed, cụ thể hóa tính lightweight bằng FLOPs/inference time, cải thiện segmentation và bổ sung Grad-CAM/XAI.